Authentication and Security

This document covers the authentication mechanisms, security practices, and data protection measures applied throughout the system.

Kiosk Authentication

Admin Password

The kiosk's admin panel is protected by a password. The password is stored as a SHA-256 hash in the `admins` table:

```
h = hashlib.sha256(password.encode()).hexdigest()
```

When a user enters a password, it is hashed and compared against the stored hash:

```
def check_admin_password(password: str) -> bool:
    h = hashlib.sha256(password.encode()).hexdigest()
    with get_conn() as conn:
        row = conn.execute("SELECT id FROM admins WHERE password_hash=?", (h,)).fetchone()
    return row is not None
```

Hashing

Hashing is a one-way function: it is computationally infeasible to recover the original password from its hash. This means that even if the database file is accessed directly, the admin password cannot be read.

SHA-256 produces a fixed 64-character hexadecimal string regardless of input length. The same input always produces the same hash, which is how password verification works.

Default Credentials

The system seeds a default admin account on first run: - Username: `admin` - Password: `admin1234`

The password should be changed through the admin panel after initial setup.

Member PIN Verification

Members can optionally have a PIN set on their profile. The `verify_pin()` function checks the entered PIN against the stored one:

```
def verify_pin(member_id: int, pin: str) -> bool:
    stored = row["pin"] or ""
    return stored == "" or stored == pin
```

A blank PIN means no restriction -- anyone can sign the member in without entering a PIN. This is a deliberate design choice for simplicity in a childcare environment where caregivers may not always have a PIN available.

Dashboard Authentication

The dashboard can be reachable over the internet (e.g. tunnelled via a reverse proxy that terminates HTTPS/TLS at its edge), so its authentication is designed to stand on its own rather than relying on being LAN-only.

Multi-User Login: Name + PIN, With Roles

Dashboard logins are no longer a single hardcoded credential. Every account lives in a `dashboard_users` table (see 02-database-design.md) with a `name`, a hashed `pin_hash`, and a `role` of either `admin` or `staff`:

```
CREATE TABLE dashboard_users (
   id          INTEGER PRIMARY KEY AUTOINCREMENT,
   name        TEXT UNIQUE NOT NULL,
   pin_hash    TEXT NOT NULL,
   role        TEXT NOT NULL CHECK(role IN ('admin', 'staff')),
   created_at  TEXT DEFAULT (datetime('now', 'localtime'))
);
```

PINs are hashed with `werkzeug.security.generate_password_hash` (PBKDF2-based, salted) rather than the kiosk's plain SHA-256 scheme -- `check_dashboard_login()` verifies with `check_password_hash()`, which is timing-safe. On first run, a single account is seeded (`name="admin"`, PIN `1234`, `role="admin"`) so the live dashboard always has a working login; a console warning is printed until that PIN is changed via the "Manage Dashboard Users" admin page.

An **admin** account is created by an existing admin from the Manage Dashboard Users page; **staff** accounts log in with just their name and PIN and can reach the web Sign In/Out page and the read-only `/status` ("Who's Here") page -- everything else (Dashboard analytics, People, Exports, Audit Log, Alerts, Manage Dashboard Users) requires `role == "admin"`.

Session Contents and Route Protection

After a successful login, the session stores the authenticated user's identity, not just a boolean:

```
session["user_id"] = user["id"]
session["name"] = user["name"]
session["role"] = user["role"]
session.permanent = True
```

Two decorators protect routes:

```
def login_required(f):        # any authenticated user, any role
   ...
```

```
    if not session.get("user_id"):
        return redirect(url_for("login"))

def admin_required(f):        # authenticated AND role == "admin"
    ...
    if session.get("role") != "admin":
        flash("Admin access required.", "error")
        return redirect(url_for("signin"))
```

`login_required` alone guards the Sign In/Out page (`/signin`), the "Who's Here" status page (`/status`), and `/logout` -- any authenticated user, admin or staff, can reach these. `admin_required` guards every other page, including editing (`/logs/sign/<id>/edit`) or adding (`/logs/sign/add`) a sign-in/out time. A staff account that hits an admin-only URL directly is redirected back to `/signin` with a flash message, rather than a bare 403.

Sessions are signed cookies -- Flask's `secret_key` cryptographically signs them, so they cannot be forged or tampered with client-side without knowing that key.

Secret Key

The Flask `secret_key` is no longer a hardcoded string committed to source. It is read from the `DASHBOARD_SECRET_KEY` environment variable (optionally via a `.env` file, loaded with `python-dotenv` if installed):

```
def _resolve_secret_key():
    key = os.environ.get("DASHBOARD_SECRET_KEY")
    if not key:
        key = secrets.token_hex(32)
        print("WARNING: DASHBOARD_SECRET_KEY is not set ...")
    return key
```

If the variable is unset, a random key is generated for that process only -- the app still starts (rather than crashing or silently using a known key), but every session is invalidated on the next restart. Production deployments should set `DASHBOARD_SECRET_KEY` so sessions survive restarts. Note that changing the secret key (or the session schema, as happened in this migration) invalidates all existing sessions -- this is expected, not a bug; see 12-build-and-deployment.md.

CSRF Protection

All state-changing (`POST`) routes are protected by `flask-wtf`'s `CSRFProtect`, applied globally:

```
csrf = CSRFProtect(app)
```

Every HTML form that performs a `POST` (login, person create/edit/delete, alerts, dashboard user management, sign-in/out) includes a hidden `csrf_token` field. Requests missing or presenting an invalid token are rejected automatically -- there is no route-by-route opt-in to forget.

Login Rate Limiting

`flask-limiter` throttles the `/login` route to 5 attempts per minute per IP address:

```
@app.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
def login():
    ...
```

This slows down credential-guessing without needing an external store -- the default in-memory limiter storage is sufficient since the dashboard runs as a single Waitress process (see below), not multiple worker processes that would each have their own counters.

Secure Cookie Configuration

```
_cookie_secure = os.environ.get("DASHBOARD_COOKIE_SECURE", "false").strip().lower() == "true"

app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
    SESSION_COOKIE_SECURE=_cookie_secure,
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),
)
```

`SESSION_COOKIE_SECURE=True` would make the browser refuse to store or send the session cookie at all over a plain HTTP connection -- safe behind a reverse tunnel, where the browser always sees an `https://` URL, but the dashboard is *also* documented as reachable directly over plain HTTP on the local network (staff/admins on the same Wi-Fi as the kiosk -- see the README and 12-build-and-deployment.md). Forcing `Secure` unconditionally broke every LAN login: with no session, the CSRF token that Flask-WTF stores in it never gets set either, so submitting the login form failed with "The CSRF session token is missing." rather than a plain "invalid password."

Since one running process serves both access paths at once, it can't statically be "Secure-only" without breaking one of them -- so this defaults to `False` (`DASHBOARD_COOKIE_SECURE` unset or anything other than `true`), which keeps LAN access working. The internet-facing path is still protected by the tunnel's own TLS termination regardless of this flag; `SESSION_COOKIE_SECURE` is a defense-in-depth extra on top of that, not the thing standing between an attacker and the login form. Set `DASHBOARD_COOKIE_SECURE=true` only for a deployment that's exclusively reachable over HTTPS (no LAN-HTTP access at all).

Production WSGI Server

The dashboard used to run on Flask's built-in development server (`app.run(...)`), which Flask itself warns is not suitable for production or internet-facing use. It's been replaced with Waitress, a pure-Python production WSGI server, in both places the dashboard is started -- standalone (`dashboard/app.py`'s `__main__` block) and embedded in the kiosk (`kiosk_signin/app.py`'s `_start_dashboard()`, which launches it on a background thread). Waitress needs no extra infrastructure (no nginx/Docker) and, as a bonus, defaults to a small thread pool rather than the dev server's single-threaded handling. See 11-threading-and-concurrency.md for how this fits the existing threading model.

SQL Injection Prevention

All database queries use parameterised placeholders (`?`) with bound parameters:

```
conn.execute("SELECT * FROM members WHERE id=?", (member_id,))
```

This is the primary defence against SQL injection. The database driver handles escaping and quoting of parameter values, making it impossible for user input to be interpreted as SQL code.

This practice is applied consistently across both `database.py` and `dashboard/app.py`. Even dynamically constructed WHERE clauses in the dashboard (for filtering logs) use parameterised queries:

```
where_parts.append("s.direction = ?")
params.append(direction)
```

Input Validation

Form Validation

Both the kiosk and dashboard validate that required fields (first name, last name) are not empty before saving. The kiosk shows a `PopupOverlay` error, while the dashboard uses `flash()` messages.

Date Validation

Export date ranges are validated using `datetime.strptime()` to ensure they match the expected `YYYY-MM-DD` format before being passed to database queries. The single-person export (`/export/person`) treats its `date_from`/`date_to` query params as optional and does not strictly validate their format -- they are only ever used in parameterised `DATE(s.timestamp) >= ?` / `<= ?` comparisons, so a malformed value just matches no rows rather than posing an injection risk.

Direction Constraint

The `sign_ins` table enforces valid directions at the database level:

```
direction TEXT NOT NULL CHECK(direction IN ('IN', 'OUT'))
```

This is a database-level constraint that rejects any value other than "IN" or "OUT".

Data Integrity

Foreign Key Constraints

Foreign keys are enabled with `PRAGMA foreign_keys=ON` on every connection. Combined with `ON DELETE CASCADE` on all foreign key definitions, this ensures:

- Deleting a member automatically removes their sign-in records, caregiver details, run sheet assignments, and holiday enrolments.
- No orphaned records can exist in child tables.

Unique Constraints

Several tables enforce uniqueness: - `admins.username` -- prevents duplicate kiosk admin accounts. - `dashboard_users.name` -- prevents duplicate dashboard logins; the "Manage Dashboard Users" page surfaces this as a friendly "already exists" error rather than a raw database error. - `run_groups.name` -- prevents duplicate run sheet names. - `holiday_enrollments(member_id, period_id)` -- prevents duplicate enrolments.

Transaction Safety

SQLite connections used with Python's `with` statement provide automatic transaction management: - Successful execution commits the transaction. - Any exception rolls back the transaction.

This ensures that multi-statement operations (e.g. creating a member and logging the event) either both succeed or both fail.

Audit Trail

All significant actions are recorded in the `audit_log` table via the `log_event()` function:

- Member and staff creation, updates, and deletion
- Sign-in and sign-out events (from both the kiosk and the web dashboard)
- Admin corrections to a sign-in/out time (`SIGN_TIME_EDITED`, recording the old and new timestamp)
- Admin additions of a missing sign-in/out time (`SIGN_TIME_ADDED`, recording the timestamp it was added as)
- Dashboard login and logout
- Dashboard user creation and deletion
- Data exports

The audit log is append-only -- there is no mechanism to delete or modify audit entries through the application. Each entry includes an event type, description, and timestamp.

This provides accountability and traceability for all system operations.

Error Handling Strategy

Silent Failures for Non-Critical Operations

Email notifications and audit logging use broad exception handling that silently catches errors:

```
try:
   # send email / log event
except Exception:
   pass
```

This is intentional for operations that should not disrupt the primary workflow. A failed email should not prevent a sign-in from being recorded.

User-Facing Errors

Operations that affect data (saving profiles, changing passwords, exporting) show explicit error messages to the user through `PopupOverlay` (kiosk) or `flash()` (dashboard).